

Java Lab Record: Exception Handling - Mobile Number Validation

Objective / Aim

To design and implement a Java program that checks whether an entered mobile number is valid, using **multiple catch blocks, built-in exceptions** (`NumberFormatException`, `ArrayIndexOutOfBoundsException`) and a **user-defined exception** (`LengthNotSufficientException`).

Problem Statement

Input a mobile number and check whether the given number is a valid mobile number or not.

- A valid mobile number is a combination of (0-9) digits of length exactly 10.
- If the given number exceeds the length of 10, raise `Invalid Mobile Number-ArrayIndexOutOfBoundsException`.
- If the given number is less than the length of 10, raise `Invalid Mobile Number - LengthNotSufficientException`.
- If the given number contains any character other than a digit, raise `Invalid Mobile Number -NumberFormatException`.

Sample Input and Expected Output

Sample Input	Expected Output
9885089465	Valid number
98567890121	Invalid Mobile Number-ArrayIndexOutOfBoundsException
88664433	Invalid Mobile Number – LengthNotSufficientException
98ab@123	Invalid Mobile Number –NumberFormatException

System Requirements

Hardware

Standard PC / Laptop
Intel Core i3 / i5 processor or equivalent
Minimum 4 GB RAM
Minimum 500 MB free storage

Software

Operating System : Windows / Linux / macOS
Java Development Kit : JDK 17 or JDK 21
Text Editor or IDE : Eclipse, IntelliJ IDEA, NetBeans, or VS Code
Command Prompt / Terminal

❏ Algorithm / Logic Description

1. Create a user-defined exception class `LengthNotSufficientException` that extends `Exception` (a checked exception).
2. Create a class `MobileNumberValidation` with a static method `validateMobileNumber(String)` that declares `'throws LengthNotSufficientException'`.
3. Keep the mobile number as a `String` (not an `int`) so that leading zeroes are not lost.
4. In `validateMobileNumber()`, check the number in this order:
 - a. If it is null OR contains any character other than 0-9
-> throw `NumberFormatException`.
 - b. If its length is more than 10
-> throw `ArrayIndexOutOfBoundsException`.
 - c. If its length is less than 10
-> throw `LengthNotSufficientException`.
 - d. Otherwise the number is valid.
5. In `main()`, read the number from the user inside a try block and call `validateMobileNumber()`.
6. If no exception occurs, print "Valid number".
7. Use three catch blocks, one for each exception, and print the required error message in each.
8. Use a finally block to close the `Scanner` in every case.

🌐 Real-time Application

Sign-up and login forms, OTP verification, banking and payment apps, and any system that must check user input before using it.

1. LengthNotSufficientException.java

💻 Source Code

```
package testexception;

// This custom class is needed because the question asks for a
// LengthNotSufficientException when the mobile number has fewer than 10
// digits.
public class LengthNotSufficientException extends Exception {

    public LengthNotSufficientException(String message) {
        super(message);
    }
}
```

```
}
}
```

1. Concept and Exception in Simple Words

Java has many ready-made exceptions, but none that means "the mobile number is too short". So we create our own. This is a **user-defined (custom) exception**.

How to make a custom exception:

1. Create a class.
2. Make it extend `Exception` (or a subclass of it).
3. Add a constructor that passes the message to `super(message)`.

Because it extends `Exception` (and not `RuntimeException`), it is a **checked exception**. The compiler forces every method that throws it to declare `throws`, and every caller to catch it or declare it too. The message given to the constructor can be read later with `getMessage()`.

Exception	Kind	Created by
<code>LengthNotSufficientException</code>	Checked	The programmer (user-defined)
<code>NumberFormatException</code>	Unchecked	Java (built-in)
<code>ArrayIndexOutOfBoundsException</code>	Unchecked	Java (built-in)

2. Step-wise Execution

1. The class `LengthNotSufficientException` is declared in package `testexception` and extends `Exception`.
2. It has one constructor that takes a `String` message.
3. When `validateMobileNumber()` runs

```
    throw new LengthNotSufficientException("...message...")
```

this constructor is called.
4. `super(message)` stores the message inside the `Exception` object.
5. The exception travels up to `main()`, where the catch block calls `exception.getMessage()` to print that stored message.

3. Algorithm

START

1. Declare class `LengthNotSufficientException` extends `Exception`
2. Declare constructor `LengthNotSufficientException(String message)`
3. Inside the constructor, CALL `super(message)`

STOP

4. UML Class Diagram

```
+-----+
|           Throwable           |
+-----+
| + getMessage():String |
```



2. MobileNumberValidation.java

Source Code

```

package testexception;

import java.util.Scanner;

public class MobileNumberValidation {

    public static void validateMobileNumber(String mobileNumber)
        throws LengthNotSufficientException {
        // Keep the input as text so leading zeroes are not lost.
        if (mobileNumber == null || !mobileNumber.matches("[0-9]*")) {
            throw new NumberFormatException();
        }

        if (mobileNumber.length() > 10) {
            throw new ArrayIndexOutOfBoundsException();
        }
        if (mobileNumber.length() < 10) {
            throw new LengthNotSufficientException("Invalid Mobile Number -
LengthNotSufficientException");
        }
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a mobile number: ");

        try {
            String mobileNumber = scanner.nextLine();

```

```

        validateMobileNumber(mobileNumber);
        System.out.println("Valid number");
    } catch (ArrayIndexOutOfBoundsException exception) {
        // The exercise specifies this built-in exception for numbers
over ten digits.
        System.out.println("Invalid Mobile Number-ArrayIndexOutOfBounds
Exception");
    } catch (LengthNotSufficientException exception) {
        // Report the custom exception used for numbers under ten digits.
        System.out.println(exception.getMessage());
    } catch (NumberFormatException exception) {
        // Reject letters and symbols instead of attempting numeric
conversion.
        System.out.println("Invalid Mobile Number -
NumberFormatException");
    } finally {
        // Close the input resource whether validation succeeds or fails.
        scanner.close();
    }
}
}

```

1. Concept and Exception in Simple Words

Key words used

throw -> actually RAISES an exception object (done inside the method).
 throws -> a WARNING in the method header: "this method may raise this
 checked exception" (here: throws LengthNotSufficientException).
 try -> the block of code that might raise an exception.
 catch -> a handler that runs when a matching exception is raised.
 One try can have MANY catch blocks (a "multi-catch" setup).
 finally -> a block that ALWAYS runs, whether or not an exception happened.

The three exceptions in this program

Condition	Exception raised	Type	Message printed
Contains any non- digit (or is null)	NumberFormatException	Built-in, unchecked	Invalid Mobile Number - NumberFormatException
More than 10 digits	ArrayIndexOutOfBoundsException	Built-in, unchecked	Invalid Mobile Number- ArrayIndexOutOfBounds Exception
Fewer than 10 digits	LengthNotSufficientException	User- defined, checked	Invalid Mobile Number - LengthNotSufficientException

Note: Normally Java raises `NumberFormatException` and `ArrayIndexOutOfBoundsException` by itself (for example when converting text to a number, or reading outside an array). Here the

program **throws them manually**, because the exercise asks for these exact exceptions and messages. Nothing is converted to a number, and no array is used.

Why the order of the checks matters

The non-digit check comes **first**. So `98ab@123` (8 characters, shorter than 10) is reported as `NumberFormatException`, not as too short. The length checks only run for numbers that contain digits only.

Why the catch order is safe

`ArrayIndexOutOfBoundsException` (a kind of `IndexOutOfBoundsException`) and `NumberFormatException` (a kind of `IllegalArgumentException`) are **not** parent and child of each other, and `LengthNotSufficientException` is a different branch again. So the three catch blocks can be in any order. If one were the parent of another, the child's catch would have to come first or the code would not compile.

Edge cases (tested by running the program)

Input	Result	Why
(empty line)	<code>LengthNotSufficientException</code> message	<code>"</code> matches <code>[0-9]*</code> (zero or more digits) and its length is 0, which is below 10
<code>98765 43210</code> (has a space)	<code>NumberFormatException</code> message	A space is not a digit
<code>0123456789</code>	Valid number	Kept as text, so the leading zero is not lost
No input at all (input stream closed)	Program crashes with <code>NoSuchElementException</code>	<code>nextLine()</code> is inside the try block, but no catch handles this exception. The finally block still closes the Scanner before the crash

On some consoles the dash in `Invalid Mobile Number - ...` can print as `?` because of the console's character encoding. The program logic is not affected.

2. Step-wise Execution

1. A Scanner object is created.
2. `"Enter a mobile number: "` is printed.
3. Execution enters the try block.
4. The user's line is read into the String `mobileNumber`.
5. `validateMobileNumber(mobileNumber)` is called:
 - a. IF `mobileNumber` is null OR does not match `[0-9]*`
-> `throw new NumberFormatException()`
 - b. ELSE IF `length > 10`
-> `throw new ArrayIndexOutOfBoundsException()`

```

    c. ELSE IF length < 10
        -> throw new LengthNotSufficientException(...)
    d. ELSE the method ends normally.
6. IF the method ended normally:
    "Valid number" is printed and all catch blocks are skipped.
IF an exception was thrown:
    the rest of the try block is skipped and Java looks for the
    first catch block that matches the exception type:
        ArrayIndexOutOfBoundsException -> prints the ArrayIndexOutOfBoundsException
message
        LengthNotSufficientException -> prints exception.getMessage()
        NumberFormatException -> prints the NumberFormatException message
7. The finally block runs in EVERY case and closes the Scanner.
8. The program ends.

```

Dry run for the four sample inputs

```

Input 9885089465    (10 digits)
  a. all digits -> no exception
  b. length 10 > 10 ? -> no
  c. length 10 < 10 ? -> no
  => prints "Valid number"

Input 98567890121   (11 digits)
  a. all digits -> no exception
  b. length 11 > 10 ? -> YES -> throws ArrayIndexOutOfBoundsException
  => caught -> "Invalid Mobile Number-ArrayIndexOutOfBoundsException"

Input 88664433      (8 digits)
  a. all digits -> no exception
  b. length 8 > 10 ? -> no
  c. length 8 < 10 ? -> YES -> throws LengthNotSufficientException
  => caught -> "Invalid Mobile Number - LengthNotSufficientException"

Input 98ab@123
  a. contains 'a','b','@' -> YES -> throws NumberFormatException
  => caught -> "Invalid Mobile Number -NumberFormatException"

```

3. Algorithm

```

START
1. Create Scanner object
2. PRINT "Enter a mobile number: "
3. TRY:
    a. READ mobileNumber (as a String)
    b. CALL validateMobileNumber(mobileNumber)
        i. IF mobileNumber is null OR not all digits:
            THROW NumberFormatException
        ii. IF length of mobileNumber > 10:
            THROW ArrayIndexOutOfBoundsException
        iii. IF length of mobileNumber < 10:
            THROW LengthNotSufficientException(message)
    c. PRINT "Valid number"
CATCH ArrayIndexOutOfBoundsException:
    PRINT "Invalid Mobile Number-ArrayIndexOutOfBoundsException"

```


Compilation and Execution

Folder structure:

```
project/  
|-- testexception/  
    |-- LengthNotSufficientException.java  
    |-- MobileNumberValidation.java
```

Compile (both files together):

```
javac -d out testexception/LengthNotSufficientException.java  
testexception/MobileNumberValidation.java
```

Run:

```
java -cp out testexception.MobileNumberValidation
```

Sample Output

Enter a mobile number: 9885089465

Valid number

Enter a mobile number: 98567890121

Invalid Mobile Number-ArrayIndexOutOfBoundsException

Enter a mobile number: 88664433

Invalid Mobile Number - LengthNotSufficientException

Enter a mobile number: 98ab@123

Invalid Mobile Number -NumberFormatException

Output:

```
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '-
-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\b
asicjava\bin' 'testexception.MobileNumberValidation'
Enter a mobile number: 9232121341
Valid number
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '-
-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\b
asicjava\bin' 'testexception.MobileNumberValidation'
Enter a mobile number: 12323435
Invalid Mobile Number ? LengthNotSufficientException
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '-
-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\b
asicjava\bin' 'testexception.MobileNumberValidation'
Enter a mobile number: 6562155524@133
Invalid Mobile Number ? NumberFormatException
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '-
-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\b
asicjava\bin' 'testexception.MobileNumberValidation'
Enter a mobile number:
Invalid Mobile Number ? LengthNotSufficientException
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '-
-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\b
asicjava\bin' 'testexception.MobileNumberValidation'
Enter a mobile number: 1321245467246868
Invalid Mobile Number-ArrayIndexOutOfBoundsException
PS C:\Users\Admin\OneDrive\Documents\basicjava> |
```

✦ Result

The Mobile Number Validation program was successfully implemented in Java using exception handling. The program used a user-defined exception, `LengthNotSufficientException`, together with the built-in exceptions `NumberFormatException` and

`ArrayIndexOutOfBoundsException`, and handled them with separate catch blocks and a finally block. It correctly displayed "Valid number" for a 10-digit input and the required error message for numbers that were too long, too short, or contained non-digit characters, matching all four sample outputs.